Course Instructors
Assoc. Prof. Mervat Abuelkheir
Dr. Mary Mohsen



MEDIA ENGINEERING AND TECHNOLOGY
Databases I · Winter 2026

MILESTONE 2
Project Description

Local Business Delivery and Fulfillment Management System

SUBMISSION
19/11/2026 (11:59 PM)

Table of Contents

1. Important Guidelines **2**
2. Background and Business Rules **3**
2.1 Background 3
2.2 Business Rules 3
2.3 Allowed Values and Constants 5
2.4 Error Codes 5
3. Database Schema **6**
4. Requirements **8**
4.1 Basic Structure of the Database 8
4.2 Basic Data Retrieval 8
4.3 Accounts: Registration, Login and Password 10
4.4 As an Admin, I should be able to: 11
4.5 As a Partner, I should be able to: 14
4.6 As a Customer, I should be able to: 15
4.7 As a Rider, I should be able to: 18
4.8 System-Level Requirements 20
5. Work Split **21**
Appendix A. Database Schema in Tabular Form **22**

1. Important Guidelines

This description is subject to minor changes that will be announced on CMS. Check CMS periodically.

Submission

- Submit one SQL file named after your team number (e.g. **Team_10.sql**) through the CMS link. It contains the database creation statement followed by every required table, procedure, function and view, in the order of Section 4.
- The file must run from top to bottom on a fresh SQL Server instance without errors. Code that does not compile is not graded.
- One submission per team. Hard deadline, no extensions, no email submissions. Cheating cases are graded zero for the whole milestone. AI tool use is prohibited and is treated as cheating.

Team work and hard requirements

- Each team has exactly 5 students. The 8 requirements marked [Hard] or [Hardest] are the hard requirements: TopSellingProducts, Suspend_Business, Reinstate_Business, Blacklist_Customer, Checkout_Cart, Cancel_Order, Search_Platform, Reconcile_Order_Totals. **Every student must implement at least 1 hard requirement, follow the Work Split from Section 5.**
- Write the name and ID of the implementing student in a comment directly above every procedure, function and view. Items without a name are not credited to any student.
- Section 5 demonstrates a fair split of the milestone on 5 students. You must use this split to avoid being penalized.

Data types and naming

- Alpha-numeric attributes use the varchar sizes given in Appendix A, and procedure and function parameters use the size of the attribute they are stored in. Every surrogate ID is int identity(1,1). Subclass tables (Admin, Partner, Rider, Customer, Restaurant, Pharmacy) use the superclass key as their primary key and have no identity column.
- Money is decimal(10,2). Quantities, ratings and minutes are int. Flags are bit. Dates are date, points in time are datetime, opening and closing hours are time. Phone numbers are char(11) and card numbers are char(16).
- Follow exactly the table names, column names, column order, data types and constraints of Section 3 and Appendix A, the names of all procedures, functions and views, the order and types of their parameters, and the order of the returned columns. Any mismatch results in a grade loss. Do not add extra tables or columns.
- Status and type columns store exactly the values listed in Section 2.3.

Input validation. The requirements describe only the business logic. Work out from the schema, Appendix A and the business rules which input values are invalid, and handle them.
Error codes. Every stored procedure returns 0 on success and returns an error code of Section 2.4 in case of any error. Every function returns an error code of Section 2.4 in case of any error, as explained there.

Implementation rules

- When a requirement says reject, or the input is invalid, the procedure must leave the database unchanged and return the matching error code of Section 2.4.
- ~~Procedures that change more than one row or table run inside a transaction so that they fully succeed or change nothing.~~
- SQL Server functions cannot modify data, so every requirement that changes data is a stored procedure.
- Unless stated otherwise, input IDs can be assumed to exist. You may write helper functions and procedures; dropAllProceduresFunctionsViews must drop them too.

- Authorization is not universally required in this milestone. For any SP/Function/View that requires authorization but no id is passed as input for authorization, assume the authorization step is delayed to MS3.

2. Background and Business Rules

2.1 Background

The platform is a local delivery marketplace, similar to popular food and pharmacy delivery apps. **Partners** own businesses (restaurants and pharmacies), **customers** order from them, **riders** deliver the orders and **admins** run the platform. The platform works in **zones** (e.g. Maadi, Zamalek). Every business, customer address and rider belongs to one zone, and every zone has a fixed **delivery fee**. A customer can only order from businesses in the zone of the delivery address.

- **Onboarding.** A partner registers a business, which stays Pending (invisible) until an admin approves it (Active) or rejects it (Rejected). A partner whose business was rejected can register it again. The partner then manages the product catalog, prices and discounts.
- Shopping. A customer saves addresses and cards, selects exactly one of them as the active address, creates a cart that is linked to the active address automatically and adds products, possibly from several businesses of the same zone.
- **Checkout.** The cart becomes an **order** that is split into one **suborder per business**. Each suborder gets a **delivery** assigned automatically to a rider of the zone. The customer pays once for the whole order using exactly one payment method and may use one **voucher**.
- **Delivery and after.** Riders mark deliveries as picked up and delivered. The customer can then rate the business (feedback) and the rider.
- **Exceptions.** A customer can cancel an order until something is picked up. An admin can suspend a business, which cancels its suborders that were not picked up, and can blacklist a fraudulent customer. Every cancellation produces a refund recorded on the payment.

2.2 Business Rules

BR-1. Users.account_status is Active or Blacklisted. Only customers are blacklisted, by an admin. A Blacklisted customer cannot log in, create carts, check out, reorder or receive vouchers.

BR-2. An admin is a Super Admin (default, Admin.zone_id is NULL) or a Zone Admin (exactly one zone; a zone may have several Zone Admins). Only a Super Admin creates admins and zones, changes zone fees, assigns Zone Admins and reinstates businesses.

BR-3. A Zone Admin may only approve, reject or suspend businesses of their zone, and only blacklist customers who have at least one address (active or not) in their zone. A Super Admin may act on any business or customer. A blacklist is lifted only by the admin who set it (BR-4).

BR-4. Blacklist. The reason and the admin of a blacklisted customer are stored in Blacklist (3.14). One admin can blacklist many customers (one-to-many), but a customer has exactly one blacklisting admin, who is the only one who can lift the blacklist (no other admin, not even a Super Admin).

BR-5. Customers only see Active businesses, and only products with is_available = 1 of Active businesses. Products are never deleted once ordered; they are deactivated instead.

BR-6. A business is open when the current time is in [opening_hour, closing_hour). If opening_hour is later than closing_hour the business works past midnight (open when now is at or after opening_hour or before closing_hour). If both are equal it is open 24 hours.

BR-7. Product.discount_amount is a fixed amount between 0 and 50% of the price (default 0). The effective unit price is price minus discount_amount.

BR-8. A customer has exactly one active address at a time (Address.is_active = 1); the customer chooses it and can change it later. A cart belongs to one customer and one of their addresses, and

new carts and orders always use the active address automatically (the customer is not asked for an address). A customer can have at most one Active cart per address, so there is only one Active cart for the active address, and only that cart can be edited or checked out. All its products must come from businesses in the zone of that address. If the active address is not in the zone of a business, the action is rejected with the zone mismatch error code. An Active cart can be edited; a Checked Out cart is frozen and is the item list of its order. CartItem.unit_price is NULL until checkout. The quantity of each order item (CartItem.quantity) is between 1 and 10.

BR-9. Pricing at checkout: SubOrder.total_price = sum of quantity times unit_price of its items. SubOrder.delivery_fee = current zone fee. Subtotal = sum of total_price. Orders.discount_amount = voucher discount on the subtotal. Orders.total_amount = subtotal + sum of delivery_fee minus discount_amount. Payment.amount = Orders.total_amount. These values never change afterwards; cancellations are recorded as refunds.

BR-10. Payment methods: Cash (paid to the rider), Card (whole amount on one card), Wallet (whole amount from the wallet). An order is paid with exactly one method; split payment is not allowed. Payment.card_id is the charged card (NULL if none). Currency is EGP.

BR-11. Orders.status is recomputed whenever a SubOrder status changes: Cancelled if all SubOrders are Cancelled; else Delivered if all non-cancelled SubOrders are Delivered; else In Progress if any SubOrder is Picked Up or Delivered; else Placed. A customer may place several orders without waiting for earlier ones to be delivered; each checkout creates a new order that starts as Placed.

BR-12. SubOrder.status goes Placed, Assigned, Picked Up, Delivered, or Placed or Assigned, Cancelled. A SubOrder is Placed when created and becomes Assigned when its Delivery is assigned to a rider. Delivery has no status: a delivery is Assigned, Picked Up, Delivered or Cancelled as its SubOrder is. An active delivery is one whose SubOrder is Assigned or Picked Up. The **least-busy available rider** of a zone is the rider of that zone with is_available = 1 and an Active account who has the fewest active deliveries (ties: smallest rider_id).

BR-13. A voucher is valid for a checkout if it belongs to the customer, is Unused and expiry_date is today or later. At most one voucher per order. Discount: if is_percentage = 1, subtotal times amount / 100 (amount from 1 to 100); otherwise the fixed amount. The discount applies to the subtotal only and never exceeds it. Used vouchers are never deleted. The Unused vouchers of a customer are deleted when the customer is blacklisted (there is no Revoked status).

BR-14. Only a Placed or Assigned SubOrder can be cancelled. The refund of one SubOrder is total_price + delivery_fee, capped so that Payment.refunded_amount never exceeds Payment.amount; cancelling a whole order refunds the remaining amount. Refunds are added to Payment.refunded_amount and, except for Cash, credited to the customer's wallet_balance. The wallet is filled only by refunds.

2.3 Allowed Values and Constants

Column or constant	Allowed values or value	Default
Users.account_status	Active, Blacklisted	Active
Admin.role	Super Admin, Zone Admin	Super Admin
Business.status	Pending, Active, Suspended, Rejected	Pending
Cart.status	Active, Checked Out	Active
Orders.status	Placed, In Progress, Delivered, Cancelled	Placed
SubOrder.status	Placed, Assigned, Picked Up, Delivered, Cancelled	Placed
Voucher.status	Unused, Used	Unused
Payment.method	Cash, Card, Wallet	No default
business_type	Restaurant, Pharmacy	No default
item_type / action	Business, Product / Add, Update, Remove	No default
Platform commission	10% of total_price of non-cancelled SubOrders	
Rider earning	80% of delivery_fee of Delivered deliveries	
Ratings	Integers from 1 to 5	
CartItem.quantity	Integers from 1 to 10 (max 10 per order item)	

2.4 Error Codes

Every stored procedure and function returns one of the codes below in case of error (Section 1).

Error code	Meaning
0	Success (stored procedures only)
-1	Invalid input: a value breaks a constraint or a business rule
-2	Duplicate: the value or record already exists
-3	Not allowed: wrong user type, out of scope, not the owner, or Blacklisted
-4	Wrong status: the current status does not allow the action
-5	Zone mismatch: the active address is not in the zone of the business
-6	Limit exceeded: ex: more than 10 of one product in the cart
-7	Payment cannot be made
-8	No available rider
-9	Nothing to process (for example an empty cart or nothing to copy)

Stored procedures return the code with RETURN. A function whose return type is int or decimal returns the (negative) code instead of its value. Login functions return a bit and return 0 on any error. Table-valued functions return an empty table on any error.

3. Database Schema

The schema is given below in parenthesis (relational) form, which is the main specification. Create the tables in this order with exactly these columns and in this column order. Underlined attributes form the primary key (PK) and italic attributes are foreign keys (FK); each FK is listed under its relation with the table and key it references. UNIQUE lists the columns (or column groups) that must be unique. The same schema is given in tabular form in Appendix A, which also states the data type, NOT NULL (NN), DEFAULT and CHECK constraints of every column. The two forms describe the same schema and Appendix A is part of this specification.

3.1 **Zone**(zone_id, zone_name, delivery_fee)

UNIQUE: zone_name

3.2 **Users**(user_id, first_name, middle_name, last_name, username, password, email, dob, account_status)

UNIQUE: username; email

3.3 **User_Phone**(*user_id*, phone)

FK: user_id → Users(user_id)

3.4 **Admin**(*admin_id*, role, *zone_id*)

FK: admin_id → Users(user_id); zone_id → Zone(zone_id)

3.5 **Partner**(*partner_id*)

FK: partner_id → Users(user_id)

3.6 **Rider**(*rider_id*, vehicle_type, plate_no, license_no, is_available, *zone_id*)

FK: rider_id → Users(user_id); zone_id → Zone(zone_id)

UNIQUE: plate_no; license_no

3.7 **Customer**(*customer_id*, wallet_balance)

FK: customer_id → Users(user_id)

3.8 **Address**(address_id, description, is_active, *customer_id*, *zone_id*)

A customer has exactly one active address (is_active = 1); all other addresses of the customer have is_active = 0.

FK: customer_id → Customer(customer_id); zone_id → Zone(zone_id)

3.9 **Card**(card_id, card_no, card_holder_name, expiration_date, *customer_id*)

FK: customer_id → Customer(customer_id)

UNIQUE: (customer_id, card_no)

3.10 **Business**(business_id, name, description, link_website, opening_hour, closing_hour, expected_delivery_time, status, suspension_reason, *partner_id*, *zone_id*)

FK: partner_id → Partner(partner_id); zone_id → Zone(zone_id)

UNIQUE: partner_id

3.11 **Business_Phone**(*business_id*, phone)

FK: business_id → Business(business_id)

3.12 **Restaurant**(*business_id*, cuisine_type, food_category)

FK: business_id → Business(business_id)

3.13 **Pharmacy**(*business_id*, pharmacy_type)

FK: business_id → Business(business_id)

3.14 **Blacklist**(*customer_id*, *admin_id*, reason)

FK: customer_id → Customer(customer_id); admin_id → Admin(admin_id)

A row exists only while the customer is Blacklisted: Blacklist_Customer inserts it and Lift_Customer_Blacklist deletes it. customer_id is the key, so a customer has at most one

blacklisting admin, while one admin can blacklist many customers (one-to-many, BR-4). Only the admin stored in admin_id can lift the blacklist.

3.15 **Product**(product_id, name, description, category, price, discount_amount, is_available, *business_id*)

FK: business_id → Business(business_id)

3.16 **Favorite_Business**(*customer_id*, *business_id*)

FK: customer_id → Customer(customer_id); business_id → Business(business_id)

3.17 **Favorite_Product**(*customer_id*, *product_id*)

FK: customer_id → Customer(customer_id); product_id → Product(product_id)

3.18 **Voucher**(voucher_id, promo_code, is_percentage, amount, expiry_date, status, *customer_id*)

FK: customer_id → Customer(customer_id)

UNIQUE: promo_code

3.19 **Cart**(cart_id, status, created_at, *customer_id*, *address_id*)

FK: customer_id → Customer(customer_id); address_id → Address(address_id)

A customer has at most one Active cart per address (one Active cart for the active address).

3.20 **CartItem**(*cart_id*, *product_id*, quantity, notes, unit_price)

FK: cart_id → Cart(cart_id); product_id → Product(product_id)

quantity is between 1 and 10 (an order item cannot exceed 10 items).

3.21 **Orders**(order_id, order_date, status, discount_amount, total_amount, *cart_id*, *zone_id*, *voucher_id*)

FK: cart_id → Cart(cart_id); zone_id → Zone(zone_id); voucher_id → Voucher(voucher_id)

UNIQUE: cart_id

Named Orders because ORDER is a reserved word.

3.22 **SubOrder**(suborder_id, status, total_price, delivery_fee, *order_id*, *business_id*)

FK: order_id → Orders(order_id); business_id → Business(business_id)

UNIQUE: (order_id, business_id)

3.23 **Delivery**(delivery_id, assigned_at, picked_up_at, delivered_at, tracking_time, rider_rating, *suborder_id*, *rider_id*)

FK: suborder_id → SubOrder(suborder_id); rider_id → Rider(rider_id)

UNIQUE: suborder_id

3.24 **Payment**(payment_id, amount, currency, method, transaction_date, refunded_amount, *order_id*, *card_id*)

FK: order_id → Orders(order_id); card_id → Card(card_id)

UNIQUE: order_id

3.25 **Feedback**(feedback_id, headline, description, rating, feedback_date, *suborder_id*)

FK: suborder_id → SubOrder(suborder_id)

UNIQUE: suborder_id

4. Requirements

4.1 Basic Structure of the Database

a) Create a database called LocalDeliveryMarketplace_Team_XX (XX is your team number).

Type: SQL Statement
Name: LocalDeliveryMarketplace_Team_XX
Input: Nothing
Output: Nothing

b) Create all tables of Section 3 (with the data types and constraints of Appendix A).

Type: Stored Procedure
Name: createAllTables
Input: Nothing
Output: Nothing

c) Drop all tables in an order that respects the foreign keys.

Type: Stored Procedure
Name: dropAllTables
Input: Nothing
Output: Nothing

d) Drop all procedures (except this one), functions and views, including helpers.

Type: Stored Procedure
Name: dropAllProceduresFunctionsViews
Input: Nothing
Output: Nothing

e) Delete all records and reset every identity so the next ID is 1.

Type: Stored Procedure
Name: clearAllTables
Input: Nothing
Output: Nothing

4.2 Basic Data Retrieval

a) All businesses (any status) with their partner. partner_name is first name and last name separated by a space.

Type: View
Name: allBusinessProfiles
Input: Nothing
Output: Table (business_id, business_name, business_type, zone_name, partner_name, status, opening_hour, closing_hour)

b) Number of Active businesses per zone, including zones with 0.

Type: View
Name: BusinessesPerZone
Input: Nothing
Output: Table (zone_id, zone_name, business_count)

c) Average rating and number of feedbacks of every business (NULL and 0 if none).

Type: View
Name: BusinessRatingsSummary
Input: Nothing
Output: Table (business_id, business_name, avg_rating, ratings_count)

d) Vouchers that are not expired and not used yet.

Type: View
Name: ActiveVouchers
Input: Nothing
Output: Table (voucher_id, promo_code, customer_id, is_percentage, amount, expiry_date)

e) Businesses awaiting approval (Pending).

Type: View
Name: PendingBusinessApprovals
Input: Nothing
Output: Table (business_id, business_name, business_type, zone_name, partner_name)

f) Every rider with the number of Delivered deliveries (0 if none) and the average rider rating (NULL if none).

Type: View
Name: RiderPerformanceSummary
Input: Nothing
Output: Table (rider_id, rider_name, zone_name, completed_deliveries, avg_rating)

g) **[Hard]** Top 10 products by units sold over non-cancelled SubOrders (ties: smallest product_id). total_revenue is the sum of quantity times unit_price.

Type: View
Name: TopSellingProducts
Input: Nothing
Output: Table (product_id, product_name, business_name, business_type, units_sold, total_revenue)

h) Per zone (the order's zone), the number of Delivered deliveries and their average tracking_time, including zones with none (0 and NULL).

Type: View
Name: ZoneDeliveryPerformance
Input: Nothing
Output: Table (zone_id, zone_name, delivery_count, avg_tracking_time)

4.3 Accounts: Registration, Login and Password

a) Register a customer (wallet_balance 0).

Type: Stored Procedure
Name: Customer_Register
Input: first_name varchar(30), middle_name varchar(30), last_name varchar(30), username varchar(30), password varchar(30), email varchar(100), phone char(11), dob date
Output: Nothing

b) Register a partner (the business is registered later in 4.5.a).

Type: Stored Procedure
Name: Partner_Register
Input: first_name varchar(30), middle_name varchar(30), last_name varchar(30), username varchar(30), password varchar(30), email varchar(100), phone char(11), dob date
Output: Nothing

c) Register an available rider in a zone.

Type: Stored Procedure
Name: Rider_Register
Input: first_name varchar(30), middle_name varchar(30), last_name varchar(30), username varchar(30), password varchar(30), email varchar(100), phone char(11), dob date, vehicle_type varchar(30), plate_no varchar(15), license_no varchar(20), zone_id int
Output: Nothing

d) Customer login: return 1 only if the credentials match a customer with an Active account, otherwise 0.

Type: Function (scalar)
Name: CustomerLoginValidation
Input: username varchar(30), password varchar(30)
Output: success bit

e) Partner login (same rules as 4.3.d).

Type: Function (scalar)
Name: PartnerLoginValidation
Input: username varchar(30), password varchar(30)
Output: success bit

f) Rider login (same rules as 4.3.d).

Type: Function (scalar)
Name: RiderLoginValidation
Input: username varchar(30), password varchar(30)
Output: success bit

g) Admin login (same rules as 4.3.d).

Type: Function (scalar)
Name: AdminLoginValidation
Input: username varchar(30), password varchar(30)
Output: success bit

h) Change the password of any user. success is 1 if the password was changed, otherwise 0 and nothing changes.

Type: Stored Procedure

Name: Change_Password

Input: user_id int, old_password varchar(30), new_password varchar(30), new_password_repeat varchar(30), success bit OUTPUT

Output: success bit (OUTPUT)

4.4 As an Admin, I should be able to:

a) Create an admin (Super Admin only). A NULL role means Super Admin.

Admin type: Super Admin only.

Type: Stored Procedure

Name: Create_Admin

Input: first_name varchar(30), middle_name varchar(30), last_name varchar(30), username varchar(30), password varchar(30), email varchar(100), phone char(11), dob date, role varchar(20), zone_id int, super_admin_id int

Output: Nothing

b) Approve a Pending business so it becomes Active. The action is rejected if the admin is a Zone Admin not assigned to the business's zone. A Super Admin can approve any business.

Admin type: Super Admin or Zone Admin (a Zone Admin only for businesses of their zone, BR-3).

Type: Stored Procedure

Name: Approve_Business

Input: business_id int, admin_id int

Output: Nothing

c) Reject a Pending business by changing its status to Rejected. The action is rejected if the admin is a Zone Admin not assigned to the business's zone. A Super Admin can reject any business.

Admin type: Super Admin or Zone Admin (a Zone Admin only for businesses of their zone, BR-3).

Type: Stored Procedure

Name: Reject_Business

Input: business_id int, admin_id int

Output: Nothing

d) **[Hard]** Suspend an Active business with a reason.

- Reject the action if the business is not Active or the admin is out of scope (BR-3).
- Set the status to Suspended, store the reason and set is_available = 0 on all of its products.
- Cancel each of its Placed or Assigned SubOrders, refund it (BR-14) and recompute the order status (BR-11). If this cancels every SubOrder of an order, restore the order's voucher as in Cancel_Order.

Admin type: Super Admin or Zone Admin (a Zone Admin only for businesses of their zone, BR-3).

Type: Stored Procedure

Name: Suspend_Business

Input: business_id int, admin_id int, reason varchar(255)

Output: Nothing

e) **[Hard]** Reinstate a Suspended business (Super Admin only, whoever suspended it). The status becomes Active, suspension_reason is cleared and all products stay unavailable until the partner re-enables them.

Admin type: Super Admin only.
Type: Stored Procedure
Name: Reinstate_Business
Input: business_id int, super_admin_id int
Output: Nothing

f) Create a zone (Super Admin only).

Admin type: Super Admin only.
Type: Stored Procedure
Name: Create_Zone
Input: zone_name varchar(50), delivery_fee decimal(10,2), super_admin_id int
Output: Nothing

g) Change a zone delivery fee (Super Admin only). Existing SubOrders keep their own fee.

Admin type: Super Admin only.
Type: Stored Procedure
Name: Update_Zone_Fee
Input: zone_id int, new_fee decimal(10,2), super_admin_id int
Output: Nothing

h) Assign or move a Zone Admin to a zone (Super Admin only).

Admin type: Super Admin only.
Type: Stored Procedure
Name: Assign_Admin_Zone
Input: admin_id int, zone_id int, super_admin_id int
Output: Nothing

i) Issue a voucher to a customer. Reject the operation if the customer is Blacklisted, the promo code already exists, or any validation or business rule is violated.

Admin type: Super Admin or Zone Admin.
Type: Stored Procedure
Name: Issue_Voucher
Input: customer_id int, promo_code varchar(30), is_percentage bit, amount decimal(10,2), expiry_date date, admin_id int
Output: Nothing

j) **[Hard]** Blacklist a customer for fraudulent behavior.

- Reject the action if the customer is already Blacklisted.
- Set account_status to Blacklisted and add the customer to Blacklist with the admin and the reason (BR-4).
- Cancel all of the customer's orders that Cancel_Order would accept, then delete all of their Unused vouchers.

Admin type: Super Admin or Zone Admin (a Zone Admin only for customers with an address in their zone, BR-3).
Type: Stored Procedure
Name: Blacklist_Customer
Input: customer_id int, admin_id int, reason varchar(255)
Output: Nothing

k) Lift a blacklist. Only the admin stored in Blacklist.admin_id, i.e. the admin who blacklisted the customer, can lift it; reject the action for any other admin. The account becomes Active and the customer is removed from Blacklist. Deleted vouchers are not restored.

Admin type: only the admin (Super Admin or Zone Admin) who blacklisted the customer (BR-4).
Type: Stored Procedure
Name: Lift_Customer_Blacklist
Input: customer_id int, admin_id int
Output: Nothing

l) For a zone and date range: number of non-cancelled orders, their average total_amount and the platform commission on them.

Admin type: Super Admin or Zone Admin.
Type: Table-Valued Function
Name: ZoneOrderReport
Input: zone_id int, from_date date, to_date date
Output: Table (zone_id, zone_name, number_of_orders, avg_order_total, platform_commission)

m) Top 5 riders by deliveries delivered in a period (ties: smallest rider_id). rider_rank is 1 to 5. Riders with none are excluded.

Admin type: Super Admin or Zone Admin.
Type: Table-Valued Function
Name: TopRiders
Input: from_date date, to_date date
Output: Table (rider_rank, rider_id, rider_name, completed_delivery_count)

n) Revenue of one day per zone and business type, over non-cancelled SubOrders of orders placed that day.

Admin type: Super Admin or Zone Admin.
Type: Table-Valued Function
Name: Daily_Revenue_Report
Input: report_date date
Output: Table (zone_name, business_type, number_of_suborders, gross_sales, delivery_fees, platform_commission)

4.5 As a Partner, I should be able to:

a) Register my business. It starts Pending. A partner owns at most one business. If my business was Rejected, registering again does not create a second business: the Rejected business gets the new data (its phones and its Restaurant or Pharmacy row are replaced) and goes back to Pending. A Restaurant needs cuisine_type and food_category, a Pharmacy needs pharmacy_type (unused inputs are NULL).

cuisine_type is the cooking style of the restaurant (for example Egyptian, Italian, Chinese or Lebanese). food_category is the kind of meal the restaurant mainly serves, whatever its cuisine (for example Fast Food, Grills, Seafood, Pizza or Desserts); an Italian restaurant, for instance, can have food_category Pizza. pharmacy_type is the kind of pharmacy (for example Retail, Hospital or Online).

Type: Stored Procedure

Name: Register_Business

Input: partner_id int, name varchar(100), description varchar(255), link_website varchar(255), zone_id int, opening_hour time, closing_hour time, expected_delivery_time int, phone char(11), business_type varchar(20), cuisine_type varchar(50), food_category varchar(50), pharmacy_type varchar(50)

Output: Nothing

b) Update my operating hours and website, and add a phone number.

Type: Stored Procedure

Name: Update_Business_Info

Input: business_id int, opening_hour time, closing_hour time, phone char(11), link_website varchar(255)

Output: Nothing

c) Add a product (product_id is NULL) or update one of my products. Reject it if the business is Suspended or the product is not mine.

Type: Stored Procedure

Name: Upsert_Product

Input: business_id int, product_id int, name varchar(100), description varchar(255), category varchar(50), price decimal(10,2), is_available bit

Output: Nothing

d) Deactivate or reactivate a product. Inactive products are hidden from customers but stay in sales history. Reject reactivation while the business is Suspended.

Type: Stored Procedure

Name: Toggle_Product_Status

Input: product_id int, is_available bit

Output: Nothing

e) Set a product discount amount (0 removes it).

Type: Stored Procedure

Name: Set_Product_Promotion

Input: product_id int, discount_amount decimal(10,2)

Output: Nothing

f) Units sold and revenue per product of my business for orders placed in a date range (non-cancelled SubOrders, sold products only).

Type: Table-Valued Function
Name: BusinessSalesReport
Input: business_id int, from_date date, to_date date
Output: Table (product_id, product_name, units_sold, revenue_per_product)

g) My average rating, number of feedbacks and count per star.

Type: Table-Valued Function
Name: BusinessFeedbackOverview
Input: business_id int
Output: Table (avg_rating, number_of_feedbacks, five_star, four_star, three_star, two_star, one_star)

4.6 As a Customer, I should be able to:

a) Add a new address. It becomes my active address and all the other addresses become inactive

Type: Stored Procedure

Name: Add_Customer_Address

Input: customer_id int, description varchar(150), zone_id int

Output: Nothing

b) Update (description and zone) or delete one of my addresses. Reject it if the address is not mine or belongs to a Placed or In Progress order. Also reject Delete if it is used by any cart.
Type: Stored Procedure
Name: Update_Delete_Customer_Address
Input: customer_id int, address_id int, description varchar(150), zone_id int, action varchar(20)
Output: Nothing

c) Set the selected address as the customer's active address, ensuring only one address is active at a time. Reject the request if the address does not belong to the customer. New carts and orders will use the active address, while carts linked to other addresses remain saved but unavailable until their address becomes active again.

Type: Stored Procedure
Name: Change_Active_Address
Input: customer_id int, new_address_id int
Output: Nothing

d) Save a card.

Type: Stored Procedure
Name: Add_Card
Input: customer_id int, card_no char(16), card_holder_name varchar(100), expiration_date date
Output: Nothing

e**)** Create an empty cart for my active address, which is picked automatically (I am not asked for an address). Reject it if I am Blacklisted, I have no active address, or my active address already has an Active cart (only one Active cart per active address).

Type: Stored Procedure
Name: Create_Cart
Input: customer_id int
Output: Nothing

f**)** Add a product with a quantity and optional note to my Active cart (the cart of my active address). If it is already there, add to its quantity and replace the note if a new one is given. . Reject it with an error message if the cart's address is not my active address or the product's business (e.g. a restaurant) is not in the zone of my active address. Reject it if the resulting quantity would exceed 10.

Type: Stored Procedure
Name: Add_To_Cart
Input: cart_id int, product_id int, quantity int, notes varchar(255)
Output: Nothing

g**)** Remove a product from one of my Active carts.

Type: Stored Procedure
Name: Remove_From_Cart
Input: cart_id int, product_id int
Output: Nothing

h**)** Return the discount of a voucher on a subtotal (BR-13), or 0 if the voucher is NULL, missing, not Unused or expired. It does not change the voucher.

Type: Function (scalar)
Name: Calculate_Voucher_Discount
Input: voucher_id int, order_subtotal decimal(10,2)
Output: discount decimal(10,2)

i**)** **[Hardest!]** Checkout a cart into an order as one atomic unit.

- Reject the whole checkout if: the cart is not Active or is empty; the customer is Blacklisted; the cart address is not the active address of the customer (the order always goes to the active address and no address is given as input); any product is unavailable, from a business that is not Active, outside the zone of the active address (return the zone mismatch error code) or closed now (BR-6) ; the voucher is not valid ; the payment cannot be made (next point);
- Payment (BR-10): the whole order is paid with one method. Card needs a valid card_id of the customer; Wallet needs enough balance; Cash needs nothing.
- Copy each effective unit price into CartItem.unit_price, create the order (zone of the address), and create one SubOrder per business (not per business type) with its total_price and delivery_fee. The new order starts as Placed (BR-11).
- Compute and store discount_amount (Calculate_Voucher_Discount) and total_amount . Mark the voucher Used.
- Create one Delivery per SubOrder and set each SubOrder to Assigned.
- Create the Payment, deduct total_amount from the wallet if the method is Wallet, and set the cart to Checked Out.

Type: Stored Procedure
Name: Checkout_Cart
Input: cart_id int, voucher_id int (nullable), payment_method varchar(20), card_id int (nullable)
Output: Nothing

j**)** **[Hard]** Cancel an order before anything is picked up.

- Reject it if the order is Cancelled or any SubOrder is Picked Up or Delivered.
- Cancel all remaining SubOrders (and so their Deliveries) and the order, and refund the remaining amount (BR-14).
- If a voucher was used, set it back to Unused and set the order's voucher_id to NULL.

Type: Stored Procedure
Name: Cancel_Order
Input: order_id int
Output: Nothing

k**)** Create a new Active cart with the same products, quantities and notes as a past order, skipping products that can no longer be ordered, for an address I choose. The chosen address must be mine and in the same zone as the original order; if it is not my active address, it becomes my active address. Reject it if I am Blacklisted, the chosen address is not mine or is in a different zone from the order, the chosen address already has an Active cart (BR-8) or nothing can be copied.

Type: Stored Procedure
Name: Reorder
Input: order_id int, address_id int
Output: Nothing

l**)** Leave one feedback for a Delivered SubOrder.

Type: Stored Procedure
Name: Submit_Feedback
Input: suborder_id int, headline varchar(100), description varchar(500), rating int
Output: Nothing

m**)** Rate once the rider of a Delivered SubOrder (stored in Delivery.rider_rating).

Type: Stored Procedure
Name: Rate_Rider
Input: suborder_id int, rider_rating int
Output: Nothing

n**)** My order history.

Type: Table-Valued Function
Name: My_Order_History
Input: customer_id int
Output: Table (order_id, order_date, zone_name, status, number_of_suborders, discount_amount, total_amount, payment_method, refunded_amount)

o**)** Add a business or product to my favorites, or remove it if it is already there.

Type: Stored Procedure
Name: Toggle_Favorite
Input: customer_id int, item_id int, item_type varchar(20)
Output: Nothing

p**)** Lists the customer's favorite products only. is_orderable is 1 if the product can be ordered now (BR-5).

Type: Table-Valued Function
Name: My_Favorites
Input: customer_id int
Output: Table (product_id, product_name, business_name, is_orderable)

q**)** [**Hard**] Return up to 10 available products from Active businesses that serve the customer's active address. Include only products the customer has never ordered and whose categories the customer has ordered from before. Prioritize most ordered categories by the customer, breaking ties by the smallest product_id.

Type: Table-Valued Function
Name: Recommended_Products
Input: customer_id int
Output: Table (product_id, product_name, business_name, category, final_price)

4.7 As a Rider, I should be able to:

a) Set my availability. Unavailable riders keep their current deliveries but get no new ones.

Type: Stored Procedure
Name: Set_Rider_Availability
Input: rider_id int, is_available bit
Output: Nothing

b) My active deliveries. (status is the SubOrder status)

Type: Table-Valued Function
Name: My_Active_Deliveries
Input: rider_id int
Output: Table (delivery_id, order_id, suborder_id, business_name, customer_name, address_description, zone_name, status, assigned_at)

c) Mark an Assigned delivery as Picked Up (its SubOrder goes from Assigned to Picked Up), stamp picked_up_at and recompute the order status.

Type: Stored Procedure
Name: Mark_Picked_Up
Input: delivery_id int
Output: Nothing

d) Mark a Picked Up delivery as Delivered (its SubOrder goes from Picked Up to Delivered), stamp delivered_at, store tracking_time in minutes since pickup and recompute the order status.

Type: Stored Procedure
Name: Mark_Delivered
Input: delivery_id int
Output: Nothing

e) My earnings for deliveries delivered in a date range.

Type: Table-Valued Function
Name: My_Earnings
Input: rider_id int, from_date date, to_date date
Output: Table (completed_deliveries, total_delivery_fees, total_earnings)

f) My average rider rating, number of ratings and count per star.

Type: Table-Valued Function
Name: My_Rating_Summary
Input: rider_id int
Output: Table (avg_rating, ratings_count, five_star, four_star, three_star, two_star, one_star)

g) My Delivered and Cancelled deliveries assigned in a date range. (status is the SubOrder status)

Type: Table-Valued Function
Name: My_Delivery_History
Input: rider_id int, from_date date, to_date date
Output: Table (delivery_id, order_id, business_name, status, assigned_at, picked_up_at, delivered_at, tracking_time)

4.8 System-Level Requirements

a) **[Hard]** Search for Active businesses by name using the provided keyword. Return only businesses that serve the customer's active address, ranked by relevance: 1 for an exact match, 2 for names starting with the keyword, and 3 for names containing the keyword. Include a priority column to indicate the match type and sort the results in ascending priority order.

Type: Table-Valued Function
Name: Search_Platform
Input: customer_id int, keyword varchar(100)
Output: Table (business_id, business_name, priority)

b) **[Hard]** Return every order whose total_amount differs from the sum of its SubOrders' total_price and delivery_fee (cancelled ones included) minus discount_amount, whose payment amount differs from total_amount, or that has no payment. Do not fix anything.

Type: Table-Valued Function
Name: Reconcile_Order_Totals
Input: Nothing
Output: Table (order_id, expected_total, order_total, payment_amount)

c) Active businesses with at least one available product in a category. same_zone is 1 for the address zone. zone_avg_delivery_time is the average tracking_time of Delivered deliveries in the business zone. Results rank by same_zone descending then zone_avg_delivery_time.

Type: Table-Valued Function
Name: Nearby_Businesses_By_Category
Input: address_id int, product_category varchar(50)
Output: Table (business_id, business_name, zone_name, same_zone, zone_avg_delivery_time)

d) Dashboard snapshot: Active businesses, available riders, orders placed today, average tracking_time of today's deliveries and feedbacks rated 2 or less today.

Type: Table-Valued Function
Name: Platform_Health_Snapshot
Input: Nothing
Output: Table (active_businesses, available_riders, orders_today, avg_delivery_time_today, low_ratings_today)

5. Work Split

The split below divides the milestone into five coherent areas of comparable effort. Three students get two hard requirements; Students 1 and 5 get one hard requirement and more regular items. Student 3 has fewer items because Checkout_Cart is the largest requirement. Student 4 should create the tables first so that everyone can test.

Student and area	Hard	Other requirements
Student 1: Accounts and admin governance	Blacklist_Customer	Customer_Register, Partner_Register, Rider_Register, the 4 login functions, Change_Password, Create_Admin, Approve_Business, Reject_Business, Assign_Admin_Zone, Issue_Voucher, Lift_Customer_Blacklist, ActiveVouchers. ~~Detect_Voucher_Abuse~~
Student 2: Partners and catalog	Search_Platform, TopSellingProducts	Register_Business, Update_Business_Info, Upsert_Product, Toggle_Product_Status, Set_Product_Promotion, BusinessSalesReport, allBusinessProfiles, PendingBusinessApprovals, Nearby_Businesses_By_Category, Toggle_Favorite, My_Favorites. ~~Recommended_Products~~
Student 3: Shopping and ordering	Checkout_Cart, Cancel_Order	Add_Customer_Address,Update_Delete_Customer_Address, Add_Card, Create_Cart, Add_To_Cart, Remove_From_Cart, Change_Active_Address,Calculate_Voucher_Discount, Reorder, My_Order_History
Student 4: Database, zones and business moderation	Suspend_Business, Reinstate_Business	Database statement, createAllTables, dropAllTables, dropAllProceduresFunctionsViews, clearAllTables, Create_Zone, Update_Zone_Fee, BusinessesPerZone, ZoneOrderReport, Daily_Revenue_Report.
Student 5: Riders, ratings and monitoring	Reconcile_Order_Totals Recommended_Products	Set_Rider_Availability, My_Active_Deliveries, Mark_Picked_Up, Mark_Delivered, My_Earnings, My_Rating_Summary, RiderPerformanceSummary, ZoneDeliveryPerformance, TopRiders, Submit_Feedback, Rate_Rider, BusinessRatingsSummary, BusinessFeedbackOverview, Platform_Health_Snapshot. My_Delivery_History

Appendix A. Database Schema in Tabular Form

This appendix gives the same 25 relations as Section 3 in tabular form, in the same order, with the data type and the constraints of every column. PK: primary key, FK: foreign key, NN: NOT NULL (all other non-key columns allow NULL). If the two forms ever appear to differ, report it on CMS.

A.1 Zone

Column	Data type	Constraints and notes
zone_id	int identity(1,1)	PK
zone_name	varchar(50)	NN, UNIQUE
delivery_fee	decimal(10,2)	NN, CHECK (delivery_fee >= 0)

A.2 Users

Column	Data type	Constraints and notes
user_id	int identity(1,1)	PK
first_name	varchar(30)	NN
middle_name	varchar(30)	
last_name	varchar(30)	NN
username	varchar(30)	NN, UNIQUE
password	varchar(30)	NN
email	varchar(100)	NN, UNIQUE
dob	date	
account_status	varchar(20)	NN, DEFAULT Active

A.3 User_Phone

Column	Data type	Constraints and notes
user_id	int	PK, FK to Users
phone	char(11)	PK

A.4 Admin

Column	Data type	Constraints and notes
admin_id	int	PK, FK to Users
role	varchar(20)	NN, DEFAULT Super Admin
zone_id	int	FK to Zone. NULL for Super Admins

A.5 Partner

Column	Data type	Constraints and notes
partner_id	int	PK, FK to Users

A.6 Rider

Column	Data type	Constraints and notes
rider_id	int	PK, FK to Users
vehicle_type	varchar(30)	NN
plate_no	varchar(15)	NN, UNIQUE
license_no	varchar(20)	NN, UNIQUE
is_available	bit	NN, DEFAULT 1
zone_id	int	NN, FK to Zone

A.7 Customer

Column	Data type	Constraints and notes
customer_id	int	PK, FK to Users
wallet_balance	decimal(10,2)	NN, DEFAULT 0, CHECK (wallet_balance >= 0)

A.8 Address

Column	Data type	Constraints and notes
address_id	int identity(1,1)	PK
description	varchar(150)	NN
is_active	bit	NN, DEFAULT 1
customer_id	int	NN, FK to Customer
zone_id	int	NN, FK to Zone

A.9 Card

Column	Data type	Constraints and notes
card_id	int identity(1,1)	PK
card_no	char(16)	NN
card_holder_name	varchar(100)	NN
expiration_date	date	NN
customer_id	int	NN, FK to Customer. UNIQUE (customer_id, card_no)

A.10 Business

Column	Data type	Constraints and notes
business_id	int identity(1,1)	PK
name	varchar(100)	NN
description	varchar(255)	
link_website	varchar(255)	
opening_hour	time	NN
closing_hour	time	NN
expected_delivery_time	int	NN, minutes, CHECK (expected_delivery_time > 0)

Column	Data type	Constraints and notes
status	varchar(20)	NN, DEFAULT Pending
suspension_reason	varchar(255)	Only while Suspended
partner_id	int	NN, UNIQUE, FK to Partner
zone_id	int	NN, FK to Zone

A.11 Business_Phone

Column	Data type	Constraints and notes
business_id	int	PK, FK to Business
phone	char(11)	PK

A.12 Restaurant

Column	Data type	Constraints and notes
business_id	int	PK, FK to Business
cuisine_type	varchar(50)	NN
food_category	varchar(50)	NN

A.13 Pharmacy

Column	Data type	Constraints and notes
business_id	int	PK, FK to Business
pharmacy_type	varchar(50)	NN

A.14 Blacklist

Column	Data type	Constraints and notes
customer_id	int	PK, FK to Customer
admin_id	int	NN, FK to Admin
reason	varchar(255)	NN

A.15 Product

Column	Data type	Constraints and notes
product_id	int identity(1,1)	PK
name	varchar(100)	NN
description	varchar(255)	
category	varchar(50)	NN
price	decimal(10,2)	NN, CHECK (price > 0)
discount_amount	decimal(10,2)	NN, DEFAULT 0, CHECK (discount_amount >= 0 AND discount_amount <= 0.5 * price)
is_available	bit	NN, DEFAULT 1
business_id	int	NN, FK to Business

A.16 Favorite_Business

Column	Data type	Constraints and notes
customer_id	int	PK, FK to Customer
business_id	int	PK, FK to Business

A.17 Favorite_Product

Column	Data type	Constraints and notes
customer_id	int	PK, FK to Customer
product_id	int	PK, FK to Product

A.18 Voucher

Column	Data type	Constraints and notes
voucher_id	int identity(1,1)	PK
promo_code	varchar(30)	NN, UNIQUE
is_percentage	bit	NN
amount	decimal(10,2)	NN, CHECK (amount > 0)
expiry_date	date	NN
status	varchar(20)	NN, DEFAULT Unused
customer_id	int	NN, FK to Customer

A.19 Cart

Column	Data type	Constraints and notes
cart_id	int identity(1,1)	PK
status	varchar(20)	NN, DEFAULT Active
created_at	datetime	NN, DEFAULT GETDATE()
customer_id	int	NN, FK to Customer
address_id	int	NN, FK to Address

A.20 CartItem

Column	Data type	Constraints and notes
cart_id	int	PK, FK to Cart
product_id	int	PK, FK to Product
quantity	int	NN, CHECK (quantity BETWEEN 1 AND 10)
notes	varchar(255)	
unit_price	decimal(10,2)	Set at checkout

A.21 Orders

Named Orders because ORDER is a reserved word.

Column	Data type	Constraints and notes
order_id	int identity(1,1)	PK
order_date	datetime	NN
status	varchar(20)	NN, DEFAULT Placed
discount_amount	decimal(10,2)	NN, DEFAULT 0
total_amount	decimal(10,2)	NN
cart_id	int	NN, UNIQUE, FK to Cart
zone_id	int	NN, FK to Zone
voucher_id	int	FK to Voucher

A.22 SubOrder

Column	Data type	Constraints and notes
suborder_id	int identity(1,1)	PK
status	varchar(20)	NN, DEFAULT Placed
total_price	decimal(10,2)	NN
delivery_fee	decimal(10,2)	NN
order_id	int	NN, FK to Orders
business_id	int	NN, FK to Business. UNIQUE (order_id, business_id)

A.23 Delivery

Column	Data type	Constraints and notes
delivery_id	int identity(1,1)	PK
assigned_at	datetime	NN
picked_up_at	datetime	
delivered_at	datetime	
tracking_time	int	Minutes from pickup to delivery
rider_rating	int	CHECK (rider_rating BETWEEN 1 AND 5)
suborder_id	int	NN, UNIQUE, FK to SubOrder
rider_id	int	NN, FK to Rider

A.24 Payment

Column	Data type	Constraints and notes
payment_id	int identity(1,1)	PK
amount	decimal(10,2)	NN
currency	varchar(3)	NN, DEFAULT EGP
method	varchar(20)	NN
transaction_date	datetime	NN
refunded_amount	decimal(10,2)	NN, DEFAULT 0, CHECK (refunded_amount <= amount)

Column	Data type	Constraints and notes
order_id	int	NN, UNIQUE, FK to Orders
card_id	int	FK to Card

A.25 Feedback

Column	Data type	Constraints and notes
feedback_id	int identity(1,1)	PK
headline	varchar(100)	NN
description	varchar(500)	
rating	int	NN, CHECK (rating BETWEEN 1 AND 5)
feedback_date	datetime	NN, DEFAULT GETDATE()
suborder_id	int	NN, UNIQUE, FK to SubOrder